

[Войти](#)

Sivchenko_translate

1 минуту назад

Сокращаем длительность компиляции проекта на Rust с 30 до 2 минут — пример с 1000 крейтов

9 мин

0

Программирование*, Rust*, Компиляторы*, Высоконагруженные системы*, SQL*

[Перевод](#)

Автор оригинала: Gerd Zellweger

Rust скор во время выполнения, а вот компилируется он не так шустро. Едва ли это удивит читателя, имевшего дело с серьёзными базами кода на Rust. В блогах сложился целый жанр постов, повествующих о том, как скостить пару секунд с **cargo build**.

В нашем проекте мы реализовали функцию, позволяющую пользователям писать прямо на SQL, определяя таким образом таблицы и представления. Под капотом этот код на SQL компилируется в код на Rust — который затем передаётся **rustc** и с его помощью компилируется в единый бинарник. В этом двоичном файле путём *пошагового приращения* все представления актуализируются по мере того, как новые потоковые данные поступают в таблицы.

Ранее мы уже предпринимали всевозможные ухищрения, чтобы ускорить компиляцию: стирали типы, активно продавливали дедупликацию кода, сокращали строки, связанные с генерацией кода. В этом мы достаточно преуспели. Но недавно мы стали осваивать работу с крупным новым клиентом (энтерпрайз), у которого оказалась масса сложного SQL-кода. Они написали при помощи наших инструментов много больших программ. Например, там был образец размером в 8562 строк на SQL, который наш компилятор в итоге превращал примерно в ~100k строк на Rust.

Поясню: в данном случае мы компилируем не какой-то массивный монолит. Речь идёт о ~100k строк сгенерированного кода на Rust. Это «семечки» по сравнению с такими проектами как ядро Linux, в котором содержится 40 миллионов строк кода — а оно успевает скомпилироваться за считанные минуты.

Однако... на компиляцию той программы, о которой здесь рассказывается (у меня на компьютере) уходило около 25 минут. Хуже того, на полную настройку конфигурации для нашего клиента уходило около 45 минут. Причём, уже после того, как мы перенастроили генерируемый код на использование динамической диспетчеризации и в основном избавились от мономорфизации.

Рассмотрим лог из нашего менеджера:

```
[manager] SQL compilation success: pipeline 0196268e-7f98-7de3-b728-0ee339e449f  
[manager] Rust compilation success: pipeline 0196268e-7f98-7de3-b728-0ee339e449f
```

Почти всё время было потрачено на компиляцию Rust. Трансляция SQL в Rust занимает примерно минуту и сорок секунд. Хуже того, сборка Rust функционально эквивалентна сборке **release** в **cargo**, поэтому она всякий раз выполняется с нуля (это касается всего кода кроме зависимостей крейта cargo, которые на описываемом этапе уже кэшированы и поэтому переиспользуются). Даже минимальные изменения в поступающем на вход SQL приводят к полной пересборке такой гигантской программы.

Разумеется, мы пробовали работать и с отладочными сборками. Если пользоваться ими, то длительность компиляции сокращается примерно до 5 минут, но на практике они неприменимы. наших клиентов интересовала фактическая производительность во время выполнения. На этапе проверки типов в коде SQL уже известно, что код на Rust скомпилируется успешно. При этом они оперируют потоковыми данными в режиме реального времени и хотят знать, какова будет сквозная задержка и фактическая пропускная способность. Отладочные сборки попросту слишком медленно работают, поэтому данные по ним способны лишь запутать.

Что здесь происходит?

Теперь поговорим о самом удручающем.

Мы пользуемся **rustc** v1.83, и, несмотря на то, что у нас 64-ядерная машина с 128 потоками, Rust практически не задействует их в работе. Это сразу же видно, если рассмотреть **htop** во время компиляции:



Всё верно. Одно ядро загружено работой на 100%, а остальные спят.

Можно инструментировать компиляцию этого крейта, передав **-Ztime-passes** к **RUSTFLAGS**(для этого требуется перекомпиляция в режиме ночной сборки). Оказывается, что большая часть времени тратится на прогоны LLVM и генерацию кода — а эти операции, как ни жаль, являются однопоточными:

```
time: 0.346; rss: 38MB -> 342MB ( +304MB) parse_crate
time: 0.000; rss: 344MB -> 345MB ( +1MB) crate_injection
time: 5.286; rss: 345MB -> 1607MB (+1262MB) expand_crate
time: 5.287; rss: 345MB -> 1607MB (+1262MB) macro_expand_crate
time: 0.091; rss: 1607MB -> 1607MB ( +0MB) AST_validation
time: 0.002; rss: 1607MB -> 1608MB ( +1MB) finalize_imports
time: 0.029; rss: 1608MB -> 1608MB ( +0MB) finalize_macro_resolutions
time: 2.382; rss: 1608MB -> 1937MB ( +329MB) late_resolve_crate
time: 0.071; rss: 1937MB -> 1938MB ( +1MB) resolve_check_unused
time: 0.138; rss: 1938MB -> 1938MB ( +0MB) resolve_postprocess
time: 2.627; rss: 1607MB -> 1938MB ( +331MB) resolve_crate
time: 0.069; rss: 1940MB -> 1940MB ( +0MB) write_dep_info
time: 0.070; rss: 1940MB -> 1940MB ( +0MB) complete_gated_feature_checking
time: 0.217; rss: 2790MB -> 2651MB ( -139MB) drop_ast
time: 3.361; rss: 1940MB -> 2353MB ( +414MB) looking_for_entry_point
time: 3.961; rss: 1940MB -> 2346MB ( +407MB) misc_checking_1
time: 6.301; rss: 2346MB -> 2007MB ( -339MB) coherence_checking
time: 44.158; rss: 2346MB -> 3061MB ( +714MB) type_check_crate
time: 18.773; rss: 3061MB -> 5024MB (+1963MB) MIR_borrow_checking
time: 4.650; rss: 5024MB -> 5241MB ( +217MB) MIR_effect_checking
time: 0.360; rss: 5243MB -> 5255MB ( +12MB) module_lints
time: 0.360; rss: 5243MB -> 5255MB ( +12MB) lint_checking
time: 0.947; rss: 5255MB -> 5254MB ( -1MB) privacy_checking_modules
time: 1.587; rss: 5241MB -> 5254MB ( +13MB) misc_checking_3
time: 0.259; rss: 5254MB -> 5249MB ( -5MB) monomorphization_collector_root
time: 54.766; rss: 5249MB -> 7998MB (+2749MB) monomorphization_collector_graph
time: 6.086; rss: 8010MB -> 8565MB ( +554MB) partition_and_assert_distinct_symbols
time: 0.000; rss: 8414MB -> 8415MB ( +1MB) write_allocator_module
time: 35.220; rss: 8415MB -> 18037MB (+9622MB) codegen_to_LLVM_IR
time: 96.733; rss: 5254MB -> 18037MB (+12783MB) codegen_crate
time: 1333.423; rss: 10070MB -> 3176MB (-6893MB) LLVM_passes
time: 1303.074; rss: 13594MB -> 756MB (-12837MB) finish_ongoing_codegen
```

```
time: 1.091; rss: 756MB -> 756MB ( +0MB) run_linker
time: 0.105; rss: 755MB -> 755MB ( +0MB) link_binary_remove_temps
time: 1.217; rss: 756MB -> 755MB ( -1MB) link_binary
time: 1.218; rss: 756MB -> 754MB ( -2MB) link_crate
time: 1.218; rss: 756MB -> 754MB ( -2MB) link
time: 1483.483; rss: 26MB -> 514MB ( +487MB) total
```

Время от времени в течение этих 30 минут Rust успеваает поднять несколько потоков — может быть, 3 или 4 — но никогда не использует машину целиком. Даже близко.

Причина понятна: распараллеливать компиляцию сложно. Но это не какой-нибудь пограничный случай; изучив его, мы со всей очевидностью обнаружили несколько вариантов, позволяющих распараллелить компиляцию в этой программе.

Замечание: любопытный читатель спросит – а что насчёт увеличения количества **codegen-units** в файле **Cargo.toml**? Удастся ли ускорить такие проходы, если поступить так? Наш опыт подсказывает, что это роли не играет. В отчётных прогонах мы использовали задаваемое по умолчанию значение **16**, но также пробовали и такие значения как **256** при заданной по умолчанию конфигурации LTO (тонкий локальный LTO). Это немного запутывало нас (да и любого, кто не является экспертом по **rustc**). Хотелось бы почитать, как объясняется эта ситуация.

Что же с этим поделать?

Мы решили, что лучше не выдавать один гигантский крейт, в котором содержится всё и сразу. Вместо этого мы откорректировали наш компилятор, преобразующий SQL в Rust, так, чтобы он распределял вывод на несколько более мелких крейтов. В каждом таком крейте заключена лишь часть логики, и они аккуратно зависят друг от друга, а также от одного вышестоящего крейта **main**, стягивающего их все.

Результаты получились блистательными. Вот то же самое представление htop, что было показано выше, но с учётом описанных изменений при компиляции:

Прекрасно. Теперь все ядра ЦП задействуются полностью и не простаивают. Это видно и в коде: теперь программа на Rust успевает скомпилироваться всего за **2 минуты 10 секунд!**

```
[manager] Rust compilation success: pipeline 01962739-79fd-7f03-bbf2-f8e29ce21e
```

Как мы это исправили?

В большинстве проектов на Rust разделять логику на десятки (или сотни) крейтов в лучшем случае непрактично, а в худшем результат подобен ночному кошмару. Но в нашем случае это был удивительно прямой путь к цели — учитывая, как наш инструмент работает под капотом.

Когда пользователь нашего инструмента пишет код на SQL, мы преобразуем этот код в граф потока данных. Узлы данного графа — это операторы, преобразующие данные, а рёбра демонстрируют, как данные перетекают между ними. Вот небольшой фрагмент этого графа:

Поскольку весь код Rust полностью автоматически генерируется из этой структуры, мы абсолютно контролировали, как именно его разделить. Каждый оператор получает собственный крейт. Каждый крейт экспортирует всего одну функцию, которая собирает один конкретный фрагмент потока данных. Все они имеют одинаковые предсказуемые очертания. Главный вышестоящий крейт `main` просто связывает их друг с другом.

```
pub fn create_operator_0097dd9de75ffef3(circuit: &RootCircuit, catalog: &mut Catalog,
    i0: &Stream<RootCircuit, IndexedWSet<Tup1<i32>, Tup5<i32, SqlString, F64, F64, Option<i32>>>>,
    i1: &Stream<RootCircuit, IndexedWSet<Tup1<i32>, Tup0>>>,
) -> Stream<RootCircuit, WSet<Tup5<i32, SqlString, F64, F64, Option<i32>>>> {
    let operator_0097dd9de75ffef3: Stream<RootCircuit, WSet<Tup5<i32, SqlString,
    Tup5<i32, SqlString, F64, F64, Option<i32>>> {
        Tup5::new(
            (*p1).0,
            (*p1).1.clone(),
            (*p1).2,
            (*p1).3,
            (*p1).4.as_ref().cloned()
        );
    };

    return operator_0097dd9de75ffef3;
}
```

Так или иначе, необходимо подобрать названия для этих крейтов. Простой и действенный метод — хешировать содержащийся в них код Rust и результат использовать в качестве имени крейта.

Так обеспечиваются две вещи:

1. У нас получаются уникальные имена крейтов.
2. Более важный момент: удаётся с необычайной эффективностью вносить пошаговые изменения в SQL

Представьте, что пользователь самую малость откорректирует код SQL. В таком случае код большинства операторов (в пределах их крейтов) окажется идентичным (хеш не изменится), и **rustc** сможет заново использовать большинство из ранее скомпилированных артефактов. Любой новый код, добавляемый в процессе изменения, приведёт к генерации нового крейта (с новым хешем)..

Итак, о каком количестве крейтов идёт речь в рассматриваемой монструозной программе на SQL?

Давайте заглянем в каталог компилятора внутри контейнера:

```
ubuntu@12e1de52de1b:~/feldera/compiler/rust-compilation$ ls crates/  
feldera_pipe_operator_000cb1599cb60b91  feldera_pipe_operator_4aab3e223e4ddcf9  
feldera_pipe_operator_004093943841ab45  feldera_pipe_operator_4ae3aa1446d98a19  
feldera_pipe_operator_004675554aea30aa  feldera_pipe_operator_4aff1d1e8d2a6a9a  
feldera_pipe_operator_008ba4153ded3848  feldera_pipe_operator_4b3575ba2e10dad3  
feldera_pipe_operator_00bee114a0d5eb4c  feldera_pipe_operator_4b5370144b5268ae  
feldera_pipe_operator_00d71fa11f791e35  feldera_pipe_operator_4b5d1c560b048f22  
...
```

И далее:

```
ubuntu@12e1de52de1b:~/feldera/compiler/rust-compilation$ ls crates/ | wc -l  
1106
```

Именно так — 1 106 крейтов!

Кажется, многовато? Может быть. Но, с другой стороны, именно благодаря такой структуре компилятор **rustc** работает гораздо эффективнее.

На этом всё?

К сожалению, не вполне. Некоторые тайны ещё остаётся разгадать. Учитывая, что мы теперь полностью задействуем 128 потоков или 64 ядра на протяжении практически всего времени компиляции, примерные расчёты показывают, что компиляция должна занимать **25 мин / 128 = 12 сек** (или, может быть, **24 сек**, поскольку гиперпотоки — это не полноценные ядра). Тем не менее, на компиляцию всего кода уходит **170 секунд**. Конечно же, на практике не приходится ожидать линейного ускорения, но **всемерно** медленнее расчётного значения — это как-то слишком. Ведь мы делаем всего лишь параллельные вызовы **rustc**, каждый из которых выполняется независимо). Аналогичные замедления наблюдаются и на ноутбуках, где гораздо меньше памяти и ядер, так что проблема актуальна не только для огромных машин.

Вот некоторые соображения насчёт того, что здесь может происходить. Интересно, выскажете ли вы ещё какие-то идеи:

- Конкуренция за аппаратные ресурсы (у системы более чем достаточно памяти, но оспариваться могут доступные кэши)
- Узкое место — это файловая система (сомнительно, поскольку мы пытались гонять эту систему и на RAM-FS, не наблюдали при этом никакой разницы. Но конкуренция может идти на уровне блокировок в коде файловой системы в ядре)
- При компиляции каждого из тысячи крейтов теперь выполняются многие шаги, которые при использовании единственного крейта амортизируются (так и есть, но замедление не происходит при компиляции с опцией `-j1`. Отдельные крейты компилируются гораздо быстрее при условии, если компилировать их друг за другом).
- Может ли узкое место возникать на этапе линковки в ситуации, когда мы добавили 1000 крейтов? Мы пользовались **gold** и установили, что общее время линковки составляет всего лишь около **7 секунд**.

Заключение

Оказалось, что можно просто изменить подход к генерации кода Rust под капотом — и масштабировать время компиляции с учётом возможностей железа, а не бороться с этим фактором. Код, на компиляцию которого ранее уходило 30–45 минут, теперь компилируется менее чем за 3 минуты, даже при работе со сложными SQL-приложениями уровня большого предприятия.

Теги: Rust, sql, оптимизация, компиляторы, высокая производительность



4K+

Охват за 30 дней

367

Карма

@Sivchenko_translate

Пользователь

Подписаться



Комментировать

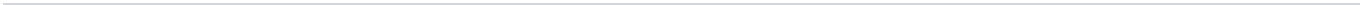
Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ

Показать лучшие за всё время





БЛИЖАЙШИЕ СОБЫТИЯ

[Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr